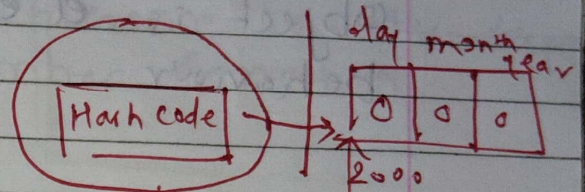


*** Java does not have garbage value**

* If the data type is numeric then default value will be zero in java and if it is non numeric (reference type) is the the value will be null

§

```
class Date {  
    int day, month, year;  
} // class date ends here
```



```
class Test {  
    public static void main(String[] args) {  
        Date d1; // reference  
        d1 = new Date();  
        System.out.println(d1);  
    }  
}
```

Handwritten note: d1.day → 0

*** In java local variable must be initialize**
E.g :-

~~class~~

```
class Date {  
    int day, month, year; } class members  
}
```

```
class Test {  
    public static void main(String[] args) {  
        int a; // local variables (inside functions)  
        Date d1 // reference  
        d1 = new Date();  
        Syso(d1);  
    }  
}
```



```
Date d2 = new Date();  
syso(d2);
```

Object :-

Object is anything which has state and behaviour

```
class Date {  
    int day, month, year;  
    String dow;
```

```
    void setDay(int d) {  
        day = d;  
    }  
}
```

*Handwritten notes: $\rightarrow // \text{setDay}(\text{int } d, \text{Date } d1);$
 $// \text{setDay}(\text{int } d, \text{Date } d1)$*

```
class Test {  
    public static void main(String[] args) {  
        Date d1 = new Date();  
        // d1.day = 9;  
        d1.setDay(9);  
    }  
}
```

Handwritten note: $\rightarrow // \text{setDay}(9, d)$

this reference

- Every member function ~~at~~ have this reference (other than static) has "this reference" which refers to current invoking object which is pass implicitly at the time of calling/calling/invoking
- The data type of this reference is ~~class type~~ class type in which member function belongs to
- for e.g. - If it is in class Date the data type of this reference is Date. If it is in class employee the data type of this reference is employee.

```
class Date {  
    int day, month, year;  
    String dow;
```

```
    void setDay(int d) {  
        this.day = d;  
    }
```

```
    void setDow(String str) {  
        this.dow = str;  
    }
```

```
class Test {
```

```
    p.s.v. main (String[] args) {
```

```
        Date d1 = new Date();
```

```
        Date d2 = new Date();
```

```
        // d1.day = 9;
```

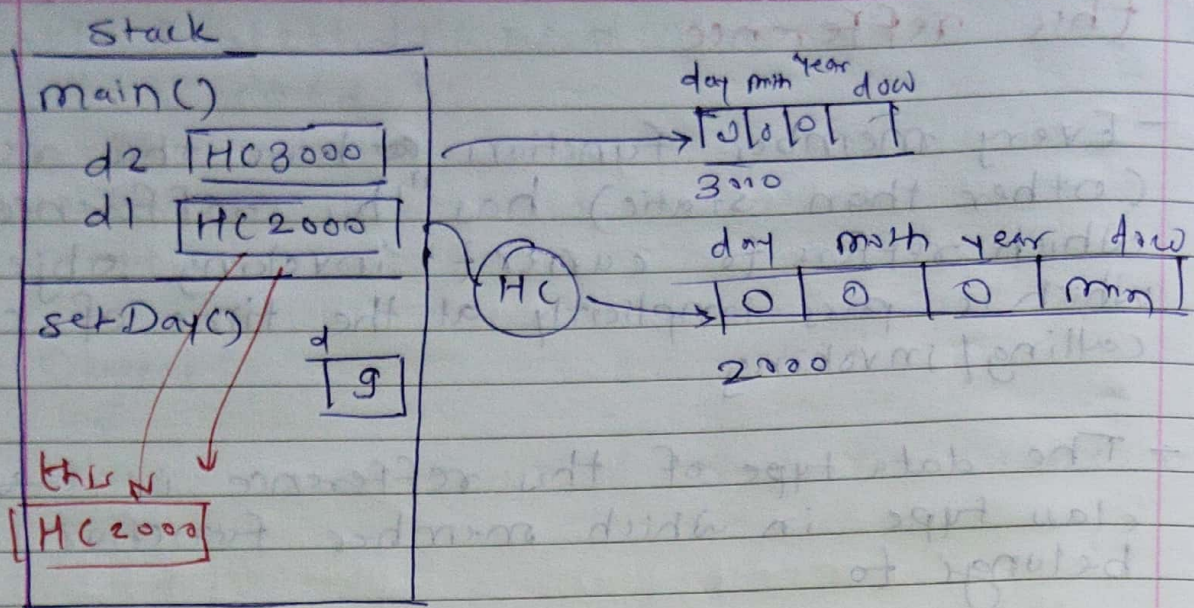
```
        d1.setDay(9);
```

```
        d1.setDow("Mon");
```

```
    }
```

→ // setDay(int d, Date this)

→ // setDay(9, d1);



* setters are also called mutators (because it can mutate)

* getters are called inspectors

* setter and getters are basic behaviour of class

11/02/24

HW

Java naming convention

★ Requirements

- Object should have initial state, right now initial is not decided by programmer
- we need something to initialize object
- we need a behavior which get called call implicitly at the time of object creation.

Constructor

- Constructors are special functions which gets called implicitly / automatically at the time of object creation.
- Constructor has a same name as a class name.
- Constructors are used to initialize an object. (Constructors are used to provide initial state to an object and setters are used to change the state i.e. to mutate an object).
- Constructor is called only once for each object.
- Constructor does not have a return type not even void.
- Constructor is always a member function.

e.g.

```
class Date {
    int day, month, year;
    string dow;

    Date() {
this.dow
        this.day = 14;
        this.month = 2;
        this.year = 2020;
        this.dow = "Sunday";
    }
}
```

```
class Test {
    p s v main (String arg)
    {
dt
        Date d1;
        d1 = new Date();
    }
}
```


Constructor

- constructors are special functions which gets called implicitly / automatically at the time of object creation
- constructor has a same name as a class name
- constructors are used to initialize an object (Constructors are use to provide initial state to change the state i.e. to mutate an object)
- constructor is called only once for each object
- constructor does not have a return type not even void
- constructor is always a member function.

E.g. :-

```
class Date {
    int day, month, year;
    String dow;
```

```
    Date() {
        this.day = 2;
        this.month = 12;
        this.year = 2025;
        this.dow = "Monday";
```

```
    }
}
class Test {
    public static void main() {
```

```
    Date d1 = new Date();
```

```
    }
}
```


Naming conventions in Java

- Naming conventions are standard rules used to name variables, methods, classes, packages and other identifiers
- They improved code readability, maintainability and consistency.

① Package naming convention

- Use lowercase letters only
- Use reverse domain name (organization structure)
- No underscores or capital letters.

e.g.:

org.apache.commons

com.mycompany.hospitalmanagement

② Class Naming conventions

- Use ~~pascal~~ PascalCase
- First letter of each word capitalized.

e.g.:

Student

EmployeeManagementSystem

③ Interface

- Use pascal case

e.g.:-

RemoteService

Runnable

④ Method naming convention

- Use camelcase (~~Camelcase~~)
- First letter lower

e.g :- methodName()
 getName()

⑤ Variable naming convention

- Use camel case

e.g :- age
 totalAmount

⑥ Constant naming convention

- Use upper ~~upper~~ ^{UPPER}
- Use UPPER-CASE
- Word separated by underscore

e.g. :- CONSTANT_NAME

⑦ Object naming

- Use camel case
- ~~usually~~

e.g emp
 student

Function overloading

- In function overloading function signature is considered in which function name must be same and parameter list must be different as follows:

- ① number of parameters must be different
- ② Data type of parameter must be different
- ③ Sequence of data type of parameter must be different

- For function overloading "name mangling" happens at compile time in java it is "name & descriptor".

What is name mangling?

→ Name mangling is a process of changing the name of overloaded function at compile time using the rule of function overloading.

- Name mangling is done by compiler
- In java name mangling kind of process is implemented through descriptors
- Descriptor usually change the name of function by using combination of package name, class name, function name, parameter list and many other things if needed

Array

```
import java.util.Scanner;

public class ArrayInJava {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();

        int[] arr; // reference
        arr = new int[n]; // creation of array

        for (int i = 0; i < arr.length; i++) {
            Syso ("Enter");
            arr[i] = sc.nextInt();
        }

        for (int ele: int arr) {
            Syso (ele);
        }
        sc.close();
    }
}
```


Static

- Static means one for a class.
- Static variable value might change according to requirement but if it changes it changes for all the objects of that particular class.
- Static variable gets memory only once at the time of class loading that's why it is also known as class variable (static variable does not get memory with every object, but it is share with every object of a particular class).

Static

- Static means one for a class.
- Static variable value might change according to requirement but if it changes it changes for all the objects of that particular class.
- Static variable gets memory only once at the time of class loading that's why it is also known as class variable (static variable does not get memory with every object, but it is share with every object of a particular class).

Without static interest rate

	account-no	Name	balance	intRate
b1	109	"Ram"	10000	10
b2	102	"Om"	5000	5
b3	107	"gaurav"	1000	12

Static

- Static means one for a class.
- Static variable value might change according to requirement but if it changes it changes for all the objects of that particular class.
- Static variable gets memory only once at the time of class loading that's why it is also known as class variable (static variable does not get memory with every object, but it is shared with every object of a particular class).

Static

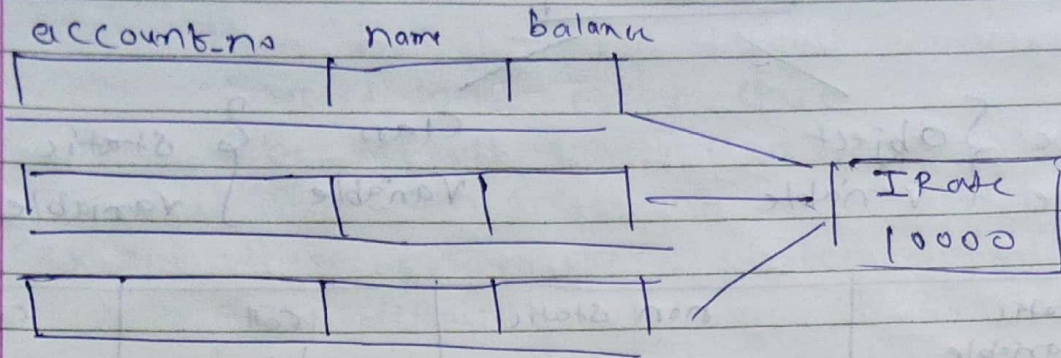
- Static means one for a class.
- Static variable value might change according to requirement but if it changes it changes for all the objects of that particular class.
- Static variable gets memory only once at the time of class loading that's why it is also known as class variable (static variable does not get memory with every object, but it is shared with every object of a particular class).

Q. 10

Without static interest rate

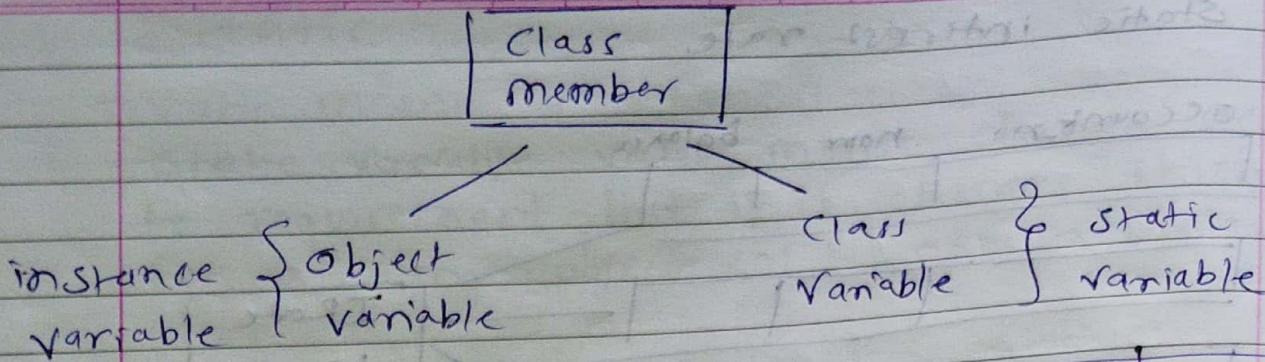
	account-no	Name	balance	interestRate
b1	109	"Ram"	10000	10
b2	102	"Om"	5000	5
b3	107	"gaurav"	1000	12

Static interest rate



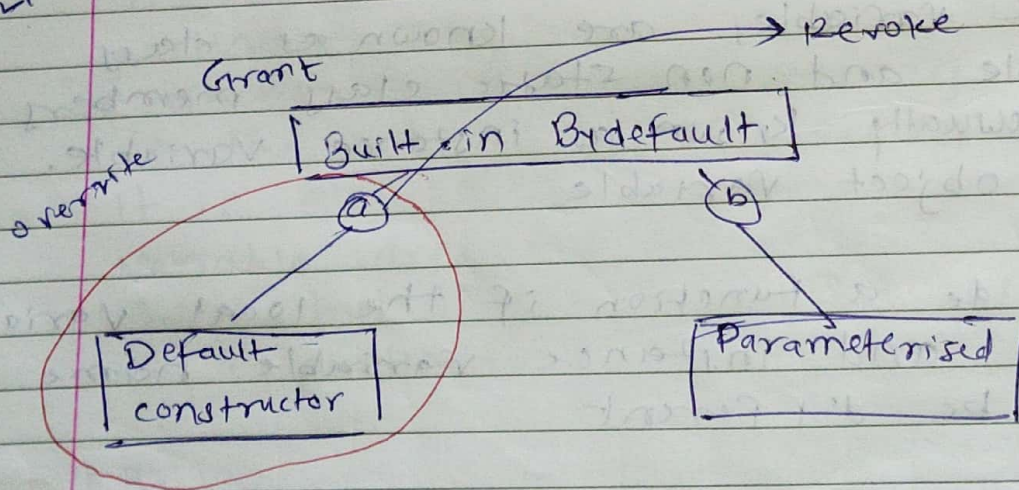
- When we declare any variable static it gets memory only once at time of class loading and it does not get memory with every object. So it is recommended that static variable should not be initialized using constructor. Because constructors are used to initialize an object.
- Static variables are known as class variable and non static class members are usually known as instance variable or object variable.
- XX - Inside a function if the local variable name and instance variable name can be different.
- But if these names are same use "this" keyword to differentiate.
- In static functions "this keyword" / ~~reference~~ reference does not exist. It is always better to keep local variable names and class member names must be different.

Why main method is static - No need to create object
 - Static blocks are given priority while class loading



	Static variable (iRate)	non static (name)	call by object	call by class Name
Static function (setInRate)	✓	X	✓	✓
non-static function (display)	✓	✓	✓	X

18/12/26



Case 1 :- When there is no written by programmer then built in by default constructor is provided with every class.
(By default constructor is granted by java)
Here we can create only default objects.

Case 2 :- When programmer writes default constructor it overwrites built in by default constructor. In this case we can ~~create~~ create only default objects i.e. Object with ~~one~~ zero parameter.

Case 3 :- When programmer writes ~~both~~ both constructors i.e. default constructor and parameterised constructor.
We can create default objects and parameterised objects respectively.

Case 4 :- When programmer do not writes default constructor but writes parameterised constructor. In this case we can create only parameterised constructor.
In this case built in by default is revoked and we cannot create zero argument objects.

** - class is known as blue print of object.

Objects Array in Java

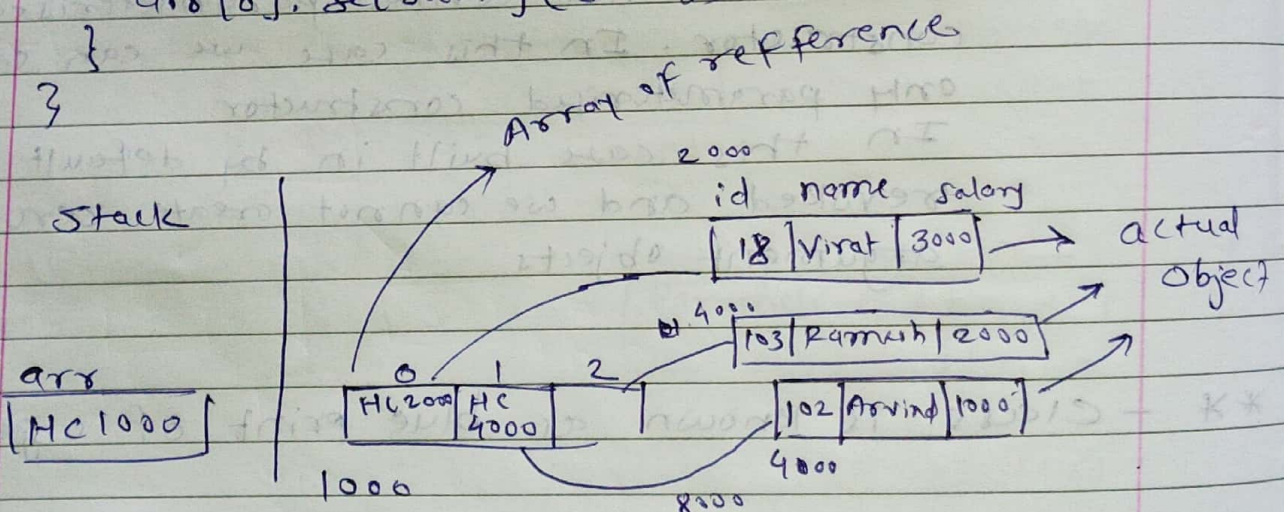
Array of object in Java

** - Array of object directly not possible in java here we need to create array of reference first. and then initiat assign them with objects.

```
class Test {
    p.s v main (String[] args) {
        Emp[] arr = new Emp[3]; // array of
                                // references

        arr[0] = new Emp();
        arr[1] = new Emp(102, "Arvind", 1000);
        arr[2] = new Emp(103, "Ramu", 2000);

        arr[0].setId(18);
        arr[0].setName("Virat");
        arr[0].setSalary(3000);
    }
}
```



Static Interview questions

Q. Why we use static block

→ To assign value to static variables.

Q. Can we use non static variables in static

→ No. because static variable get memory during class loading while a non static get memory at object level.

Q. Can we use non static variable in static function

→ No. Because object non static variable initialize or get memory at object creation.

Q. Can we use static variable in non static function

→ yes.

Inheritance

* * - In inheritance whenever we create derived class object always base class object is gets created implicitly (base = super) (derived = child & subclass)

- It is logically mandatory to write super keyword and pass parameter arguments in it. If we write super keyword without parameter or not write super keyword in parameterized constructor of derived class it calls default constructor.

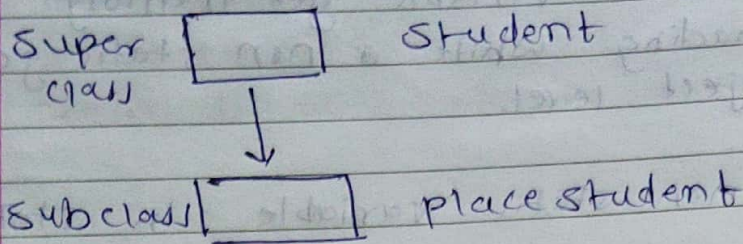
Types of inheritance

- ① simple inheritance / single inheritance
- ② Multilevel inheritance
- ③ hierarchical inheritance
- ④ hybrid inheritance

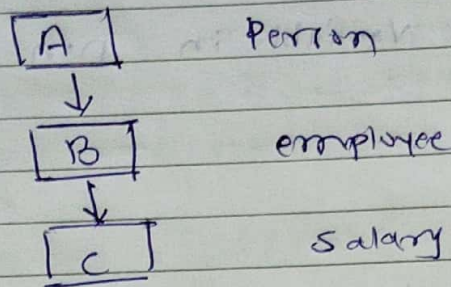
XX - Java does not allowed multiple inheritance

* Types of inheritance :-

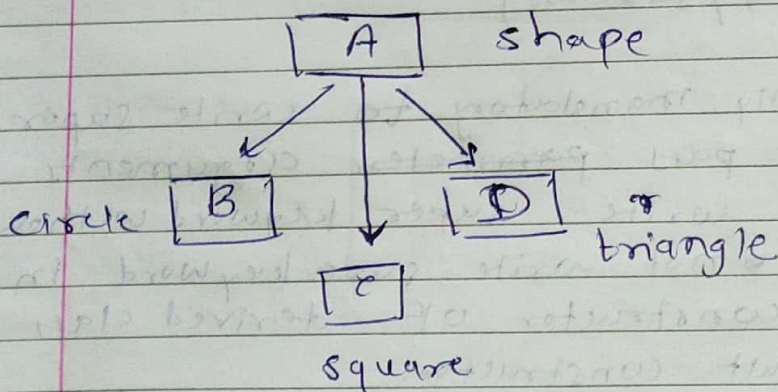
① simple inheritance



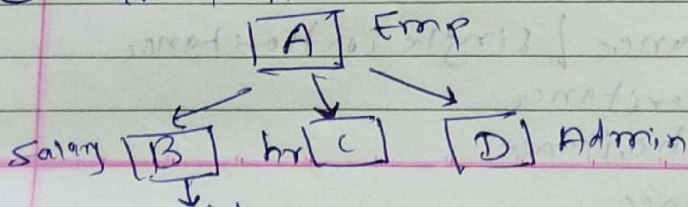
② Multilevel inheritance



③ hierarchical inheritance



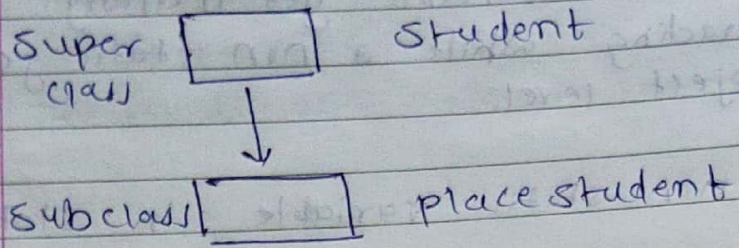
④ Hybrid inheritance



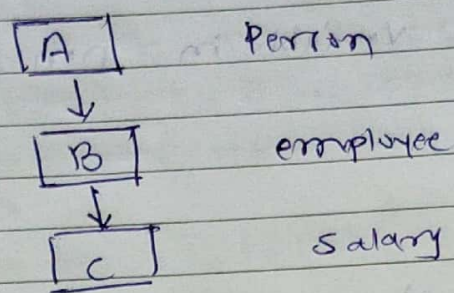
** - Java does not allowed multiple inheritance

* Types of inheritance :-

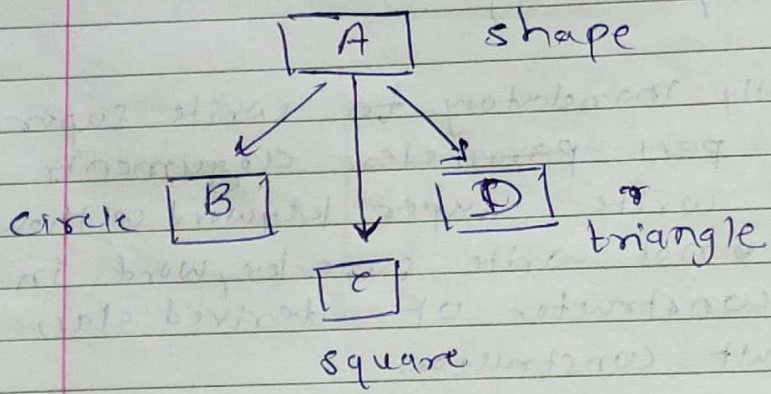
① simple inheritance



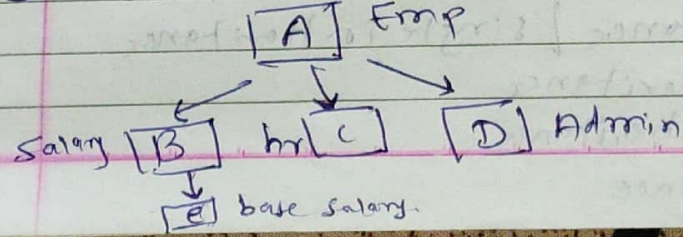
② Multilevel inheritance



③ hierarchical inheritance



④ Hybrid inheritance



- Scanned with OKEN Scanner
- DATE _____
PAGE _____
- ~~Scp~~ If super class does not allowed default constructor or object then subclass have to follow rule.

Polymorphism

* Upcasting :-

- Upcasting done when reference of generalize class & object of specialize class

* Polymorphism definition

Same message given to ~~give~~ generalize thing for generalize ~~behaviour~~ same behaviour but implemented differently.

* How to implement true polymorphism in java?

- True polymorphism is implemented through function overriding

* How to implement function overriding?

→ There are two steps in function overriding:

① Write / define / declare a function in super class.

② Define a same function in subclass with different implementation.

(Here same mean the declaration / prototype of function must match)

* How to implement true polymorphism in Java

→ True polymorphism is implemented through function overriding

* How to implement function overriding?

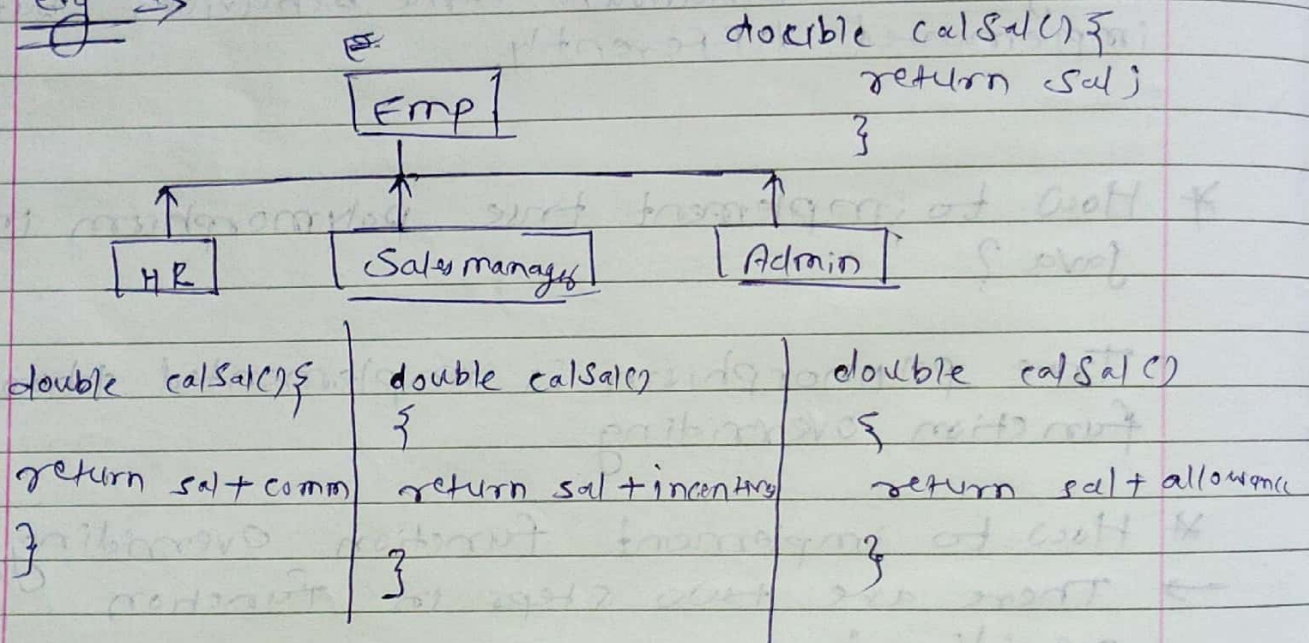
There are two steps in function overriding:

① Write/define/declare a function in super class

② Define a same function in subclass with different implementation

(Here same means the declaration/prototype of function must match)

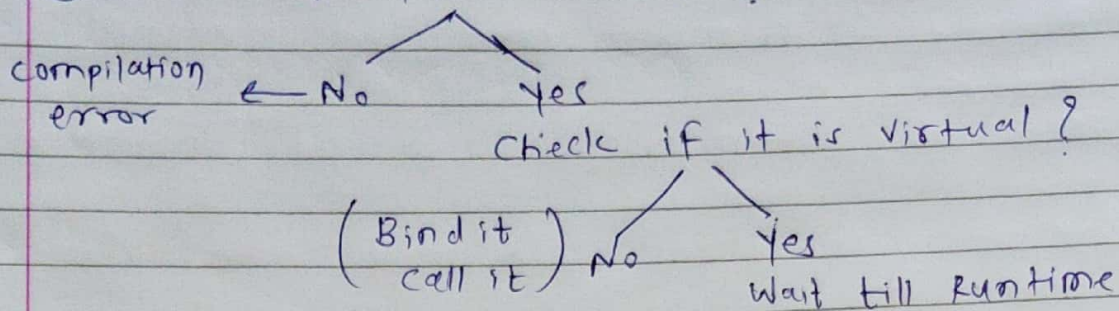
e.g. →



Pl. new Admin();
Pl. display();
Pl. callSales();

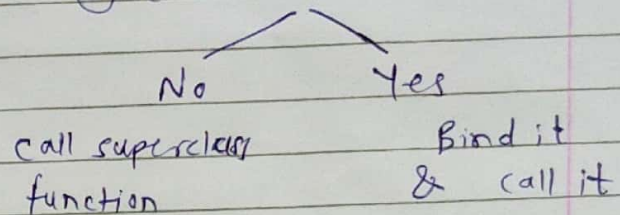
Generic Reference algorithm

- ① check the type of reference
- ② Go to that reference class
- ③ check if function exists?



at runtime

- Ⓐ check the type of object
- Ⓑ Go to that objects class
- Ⓒ check if function exists?



** Object class in java

- every generic class in java extends Object class.
- Object class also called cosmic super class.

- ** - Concrete class $\Rightarrow$ The class whose object is ~~not~~ created
- ** - Abstract class $\Rightarrow$ The class whose object is not created.

Abstract class

- A class which can not be instantiated called Abstract class
- Where type exists but instance does not exists
- A class for which we can create ref reference but can not create object

★ Abstract class

- A class which can not be instantiated called Abstract class
- Where type exists but instance does not exists.
- A class for which we can create reference but can not create object

Concrete classes

- Non abstract class for which we can create object are also known as "Concrete classes".

Abstract function

Abstract function is a function which has declaration only and does not have a definition.

Note:- concrete ^{class} ~~function~~ can not have abstract function but abstract ~~function~~ class can have concrete ~~point~~ + abstract functions.

* Note:- There is no abstract variables, only class and function can be abstract.

** - Rule:-

① ~~Abstract function must be exists in~~
abstract class

① Abstract function is possible within a abstract class

② Concrete class can not have abstract function.

③ Abstract class can have any number of abstract function and concrete function.

** - Note:- Consider abstract class ~~vehicle~~ vehicle which is base class and have derived classes as car and bike. Base class contain abstract functⁿ like breakers(). It is compulsory this functⁿ overridden in concrete class (derived)

** - It is said that abstract class must be inherited and abstract function must be overridden

Polymorphism

① Overloading

① Overriding

② false

② True

③

✓ 1) Same message

✓

③

✗ 2) given to generalize things

✓

✓ 3) for same behaviour

✓

✓ 4) But implemented differently ✓

④ Signature

④ Declaration/prototype

⑤ Compile time

⑤ Run time

(Name mangling)

Virtual by default

⑥ early binding

⑥ Late binding

(1a))

Interface

- Interface provide is-like-a relation
- In interface all functions are public and abstract by default that's why it is known as ~~pure~~ "pure abstract class".
- To maintain is like a relationship java has given keyword ~~is~~ "implements" with ~~inter~~ interface.
- following is an example of interface and the implementation

```
interface vehicle {
    public abstract void ride();
    abstract void turn();
    void stop();
}
```


interface Vehicle

{

public abstract void mdc();

abstract void turn();

void stop();

}

interface Toy

{

void play();

}

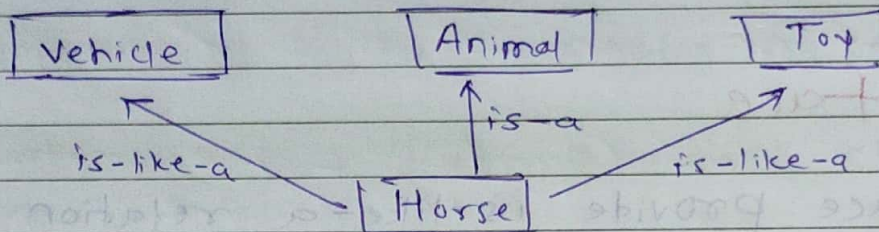
class Horse implements Vehicle, Toy

{

@ Override

}

** - Following is a hierarchy considered to explain the sequence / priority between is-a and is-like-a relationship



class Horse extends Animal implements Toy, Vehicle

{

}

- In above example extends represents is-a relationship and i.e. strongest relationship that why it must be first and implement represents is-like-a that's why it comes later.

- class Interface cannot implement interface
- class cannot extends interface.
- A class can extends only one class and multiple interfaces.

XX - We can create reference of interface but can not create object of interface.

- Whenever we implement interface "Generic reference algorithm" work as it is
- for e.g: In previous hierarchy all following combinations are possible.

i] Vehicle v1 = new Horse();

ii] Animal a1 = new Horse();

iii] Toy t1 = new Horse();

~~Why we used Interface~~

- To connect totally different h

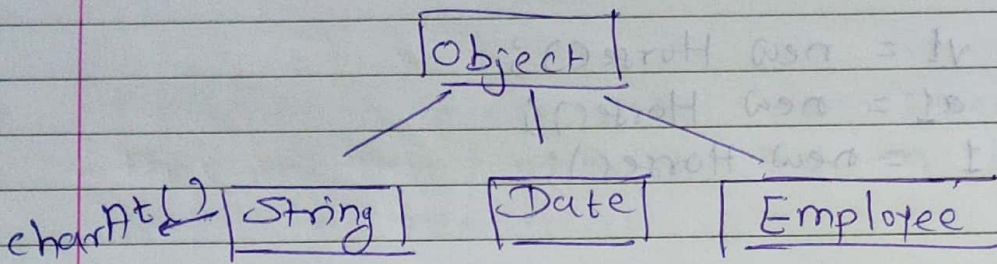
- Interface is used for following reasons:

- ① Interface provides is-like-a relationship
- ② Interface provides a ~~a~~ unbreakable contract between interface and class.
- ③ ~~flexib~~ provide ~~per~~ flexibility and multi-type compactibility.
- ④ connect totally different ~~h~~ hierarchie

Down Casting

- In OOP concepts many times upcasting happens by default i.e. reference of Superclass class is referring to object of specialized class, in this case through generic reference we can access polymorphic functions and generic functions but we can not access specialized function here, we need to explicitly down cast and then access the specialised function

example 8-



① Object o1 = new String("Om");

② String str = (String) o1;

③ str.charAt(1);
~~str.charAt(4);~~

- In first line upcasting have been implemented
- In second line we have to downcast it into String type to call specialized function charAt()

In the same way following is a similar example of employee hierarchy, as is known to us.

Note:- Below in blind down casting

```
Emp e1 = new sm(---);
```

```
sm s1 = (sm) e1;
```

```
s1.getIncentives();
```

- But in above example the way we did down casting i.e. known as blind casting which is also considered as unsafe casting because in this case compilation is always successful even if e1 is referring to any Specialized other specialized class object.
- The solution for this is safe casting which is done through "instanceof keyword" internally does RTTI
- Because it internally does ~~RTTI~~ RTTI RunTime type Identification
- which internally checks the type of object and a class mentioned in instanceof clause.
- If it matches it returns true else it returns false.
- We shall downcast accordingly.
- ** - If we do not use "instanceof" keyword then it is possible that because of blind casting "class cast exception" may occur

Note: - Java packages are folders.
O.S. Folder are not packages.

Passages

Packages

- The concept of package is use in two ways.
 - ① Naming Convention
 - ② Access Specifier.

- when we apply packages the name of class under it changes to "packageName.className".
so if package is p1 then class under it is name as Test so fully qualified name becomes p1.Test.

** - Package line must be the first line in the source code.

** - All the packages in java represents folders in O.S.

- But all the folders of O.S. are not java packages.

Public class

- We use public accessifier before class name, we use if we want to use this class in another class.

Public
Private
Protected

- By making class public the attributes and methods inside class not become public.

Syntax :

```
public Date {  
    //code  
}
```

- If we make class as public then file name and class name (public class) should be same.

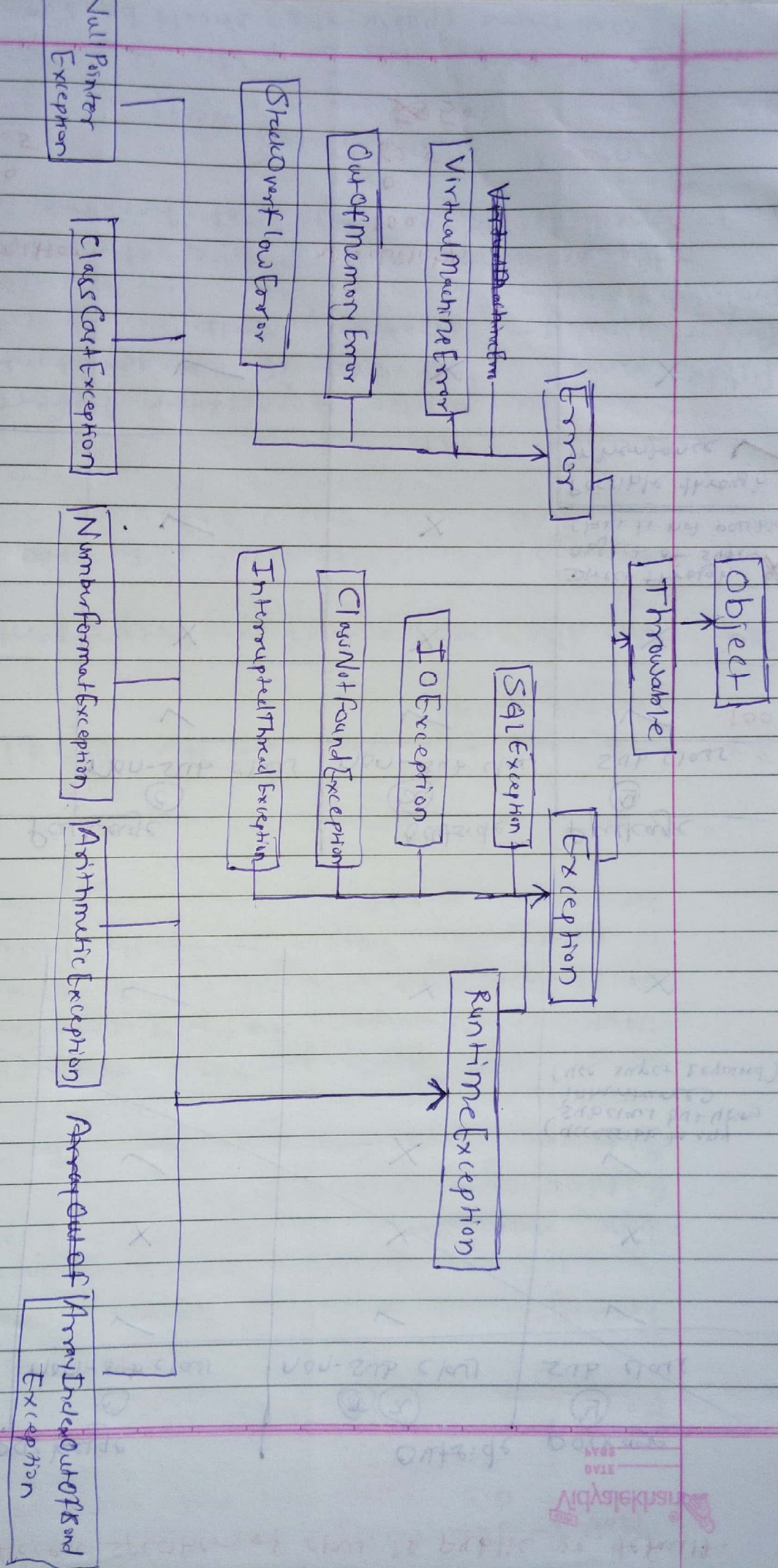
Note: Access specifier of class is public or default.

Vidyalokhan

	Same package		Outside package	
	(B) Subclass	(C) non-sub class	(A) (X) non-sub class	(Y) sub class
Public	✓	✓	✓	✓
Private	X	X	X	X
Protected	✓	✓	X	✓ (accessible to any subclass but using inheritance) (use super keyword)
(Package) (Package) (package)	✓	✓	X	X

	Same Package		Outside package	
	(B) Sub-class	(C) non-sub class	(A) non-sub class	(Y) Sub class
public 100	✓	✓	✓	✓ 100
private 0	X	X	X	X 0
Protected 62.5	✓	✓	X	Direct through Object of super class is not possible Possible through inheritance ✓
(Package) (package) 5	✓	✓	X	X

	Protection	Visibility
Public	0	100
Private	100	0
Protected	37.5	62.5
Package	0	50



Exception handling

In java almost all the exceptions execution which happen through method is binded dynamically / Runtime, so java developers were well aware of many Runtime issues which may occurs and desconnet the programming abruptly.

- So they built a very good mechanism known as exception handling framework which contain multiple ~~clases~~ classes and methods.

It also provides five keywords as follows

- ① try
- ② catch
- ③ finally
- ④ Throw
- ⑤ throws

① try :

This block is use to write a business logic where mostly user interactions, runtime input readings & runtime resource happens which might produce exception.

Exception thrown by try block is handle by relevant catch block.

Try block is always followed by a catch block (other than try with resource).

② catch

catch block handles / receives any throwable object. A catch block receives / handles anything which "is-a" Throwable.

catch block can receive Throwable Throwable and its subclass object.

We can not write a catch block without a try.

We can write multiple catch blocks which helps to provide customize error message.

When we write multiple catch blocks generalize catch block should be written at the end and specialize should be written at the beginning in the try-catch ladder.

③ throw

- throw is used to throw any ~~the~~ Throwable object explicitly "is-a" Throwable.

④ finally

- It is used to execute any clean-up code which must be executed in any condition, i.e. whether program executes smoothly or it throws an exception.

- finally always gets executed.

ex - In exception handling either try block gets

executed completely or catch block gets executed if ~~exception~~ exception occurs.

⑤ throws

- throws keyword is used to declare an inform calling function about the expected problem/exception in the called function.
- Following is the basic to use try-catch throw mechanism

```
try {
    // business logic / risky code
    if (problemFound) {
        throw new "is-a" Throwable;
    }
}
catch ("is-a" Throwable) {
    // exception handling message
}
```


Robust $\Rightarrow$ difficult to fail.

** - Exception means runtime problems

- Exception handling provides user friendly messages

- We can throw anything "is-a" Throwable (Throwable and its subclasses) and we catch only the same thing

** - Note that every object is not throwable

```
try {
```

```
    // Business logic
```

```
    // If problem occurs
```

```
    // throw object of problem
```

```
    {  
        catch (Throwable & subclass)
```

```
    {
```

```
    }
```

Throwable &
Subclasses

* Why we use try-catch block

- To separate the business logic & problematic code

Example:-

```
package p1;
```

```
public class TestException {  
    public static void main() {
```


try {

int a = Integer.parseInt(args[0]);

int b = Integer.parseInt(args[1]);

System.out.println(a/b);

}

catch (ArrayIndexOutOfBoundsException ae) {

System.out.println("please enter at least two inputs");

}

catch (ArithmeticException a) {

System.out.println("please ~~provide~~ provide second argument as zero");

}

catch (NumberFormatException nf) {

System.out.println("Enter input only integers");

}

catch (Exception e) {

System.out.println("oops something went wrong");

}

}

Notes - generalize block must be add at end of try-catch ladder

specialize block always be above written above generalize catch block in try-catch ladder

- Either try block run successfully or catch

★ Java forces checked exceptions at compile-time to handle it.

Vidyalekhan

DATE _____
PAGE _____

★ ★ — when we handle exception by try catch either try block will run successfully or catch block will execute.

★ Finally

- Finally block is written so that the code which needs to be executed in any case is written in finally. Usually ~~the code~~ is written in finally. Usually the code for resource closing is written in finally block.

★ different combinations of

- ① try - catch
- ② try - catch - catch
- ③ try - catch - finally
- ④ try - finally

★ ★ — try must be ~~write~~ followed by catch block or finally

★ Customize / user define exceptions :

* Difference between throw and throws

① throw is always part of method definition

② We throw object using throw

③ Use to throw an exception manually or explicitly.

④ Can throw only one exception

⑤ "throw" is mostly used to throw user-define or customize exception

① throws is always part of method declaration

② We use class Name with throws

③ Use to declare exception in method signature.

④ can declare multiple declaration. exception

Customize exceptions

are usually created by extending exception class so that with the whole ecosystem of exception handling can be used.

When creating customize exception you can extend it by ~~throw~~ Throwable, Exception

or RuntimeException
But extending by
Exception is recommended

Note:-

"e.printStackTrace()" it print must be remove from the finalise version of code

x Diff. between checked & unchecked.

checked

- ① All the subclass of Exception (other than RuntimeException)

- ② ~~computer~~ java enforces to handle them either by ~~try~~ writing try-catch block or adding throws clause

- ③ These problems outside java environment

- ④ Java has not written try-catch block for these exception

- ① All the subclasses of RuntimeException

- ② Java does not enforce handling

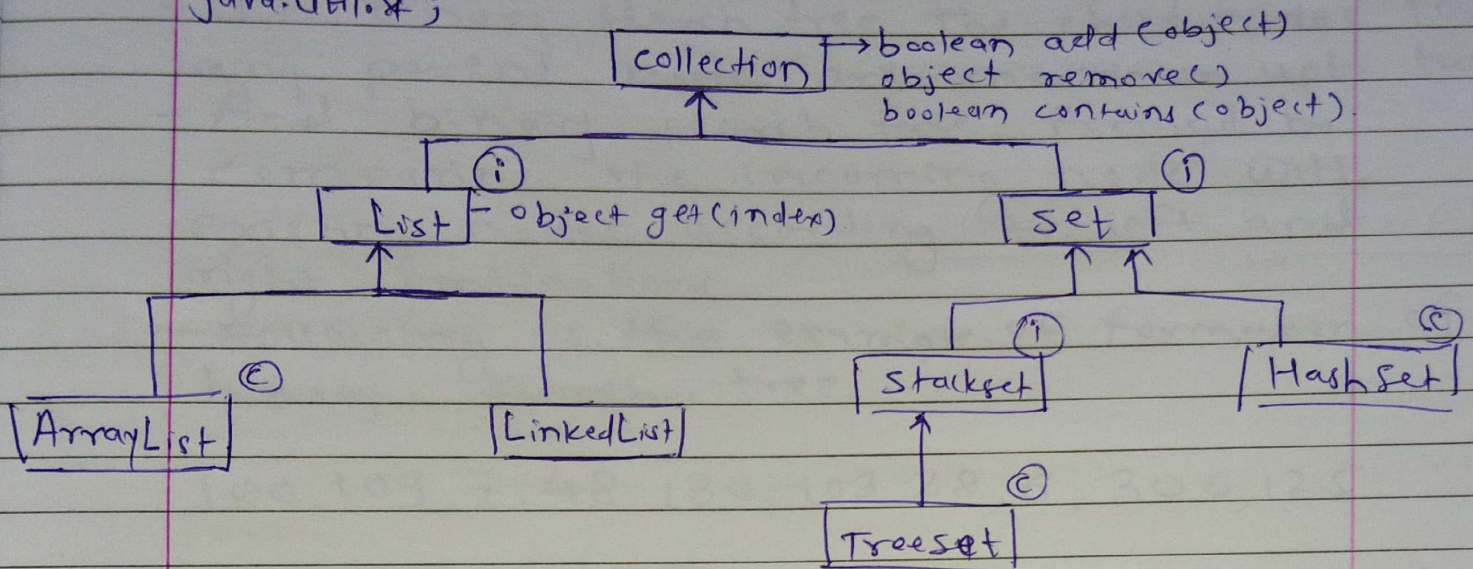
- ③ These programming/runtime issues handle by JVM.

- ④ Java has already written try catch block for these exception

Collection

- Built-in "Data structures" in java

java.util.*;



List

- Linear
- indexed
- ordered (user input)

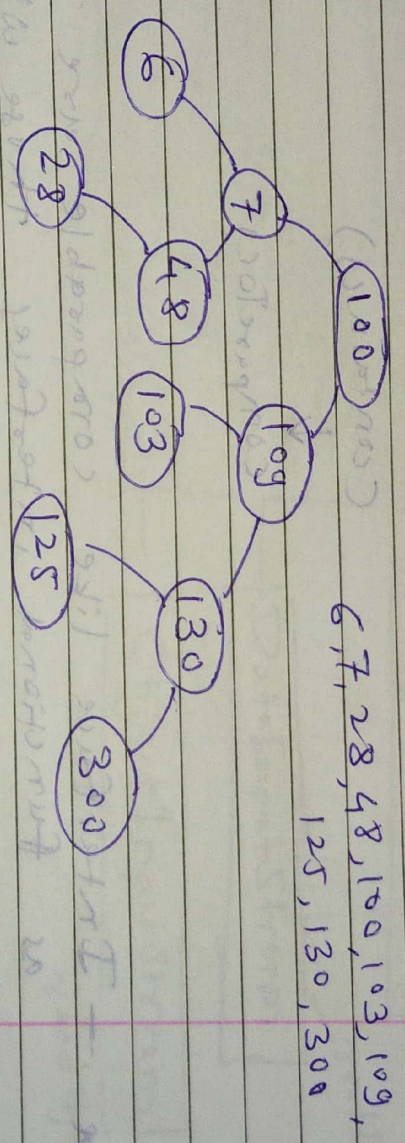
Set

- Non-linear storage
 - un-indexed
 - un-ordered
- (in this ~~use~~ set ~~use~~ input is sorted in SortedSet).

Tree set

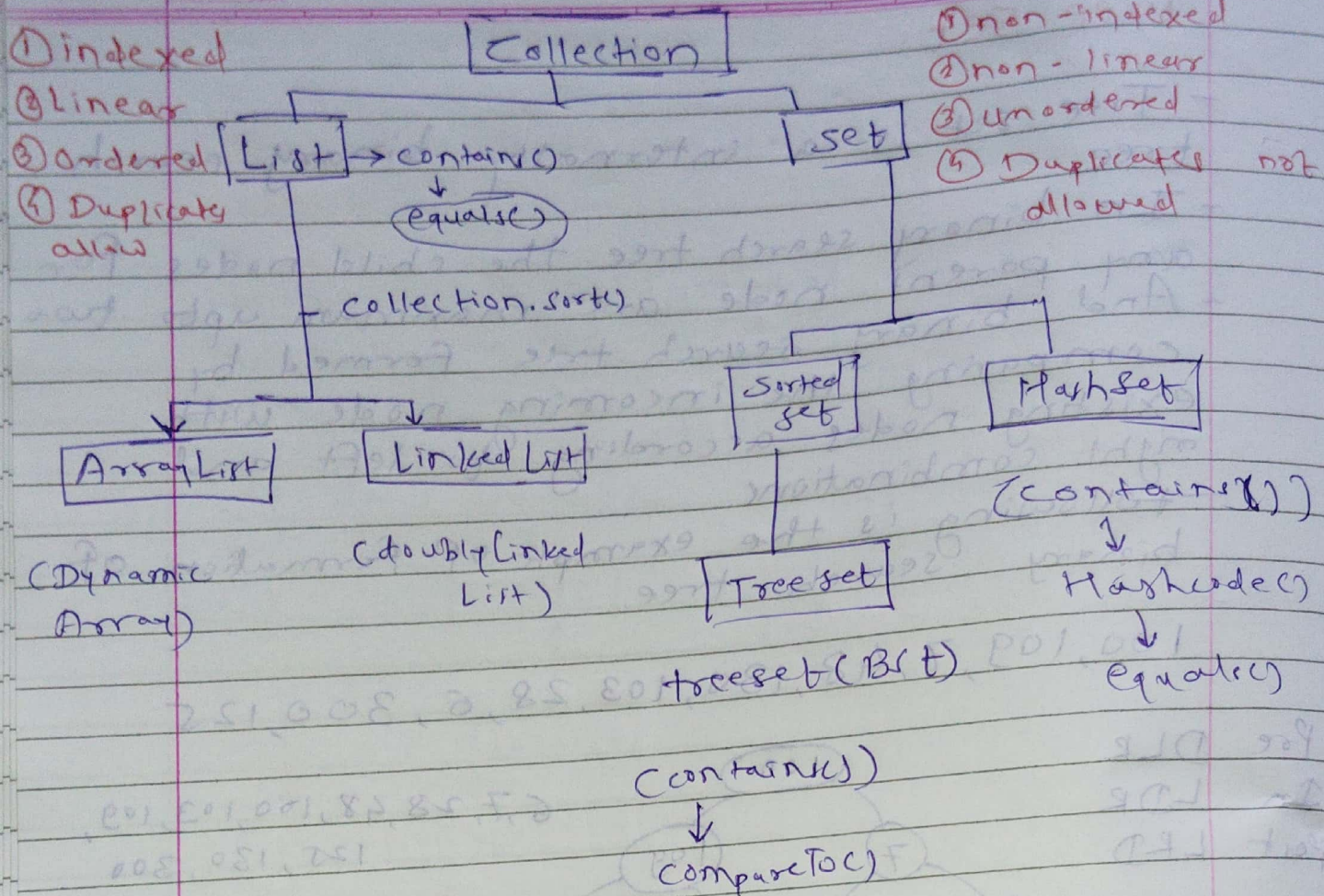
- Tree set is a internally binary search tree
- In binary search tree the child nodes for any parent node are maximum upto two
- And binary search tree formed by comparing the incoming node with existing nodes according to left and right combinations.
- Following is the example of formation of binary search tree.

100, 109, 7, 48, 130, 103, 28, 6, 300, 125



Pre DLR
In LDR
Post LRD

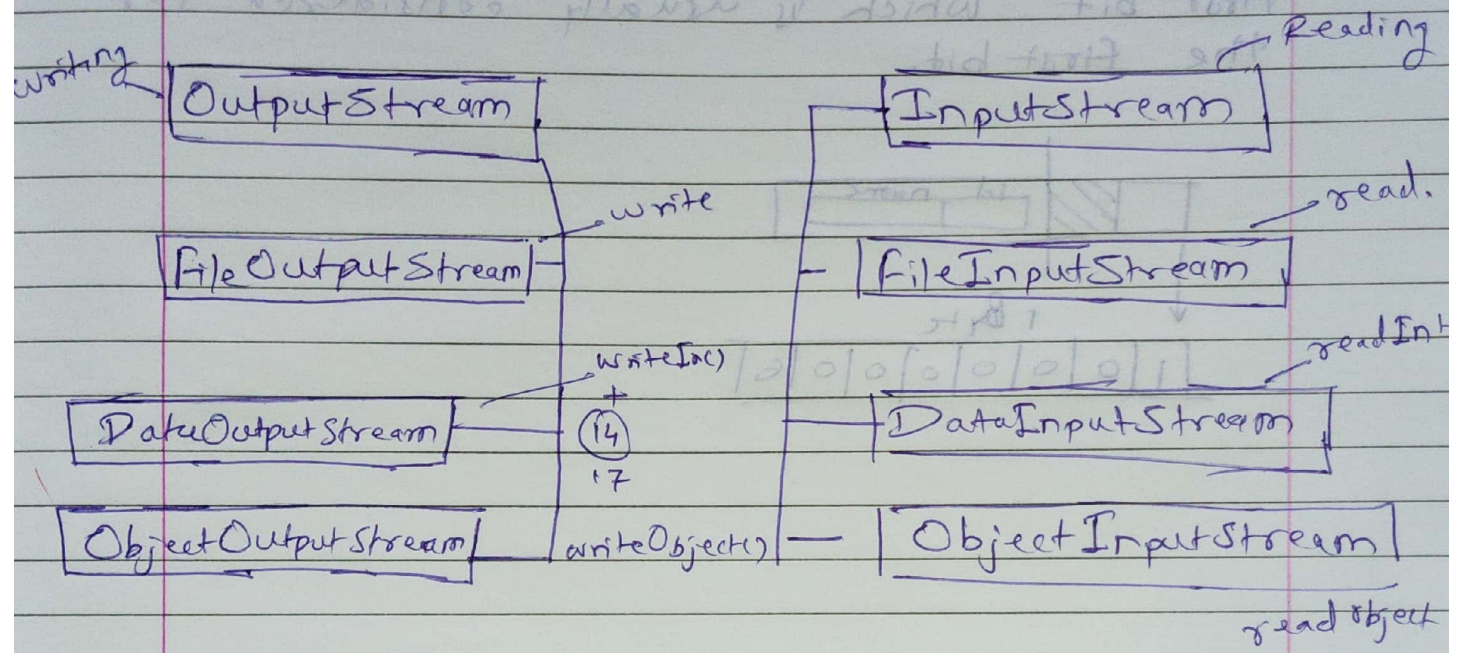
- Tree data structure formation is a non linear data structure so here three traversal pre-order, post-order, in-order depending on the data.
- In which if it is binary search tree (bst) in order traversal is always sorted



Interface like comparable are known as functional interfaces those who are having only one function.

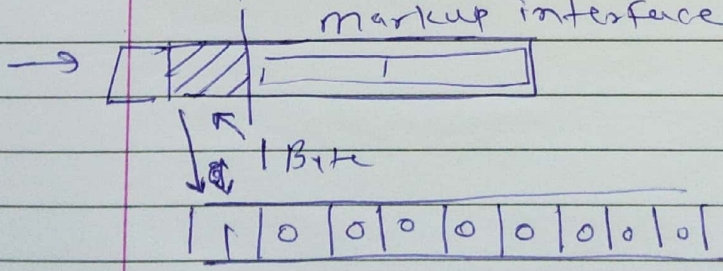
Input → Read
Output → Write

- File handling is about permanent storage and Reading & Writing.
- Here reading is considered as input and writing is considered as output.



Q. what is markup interface
→ Empty interface.

Q. why markup interface
markup interface

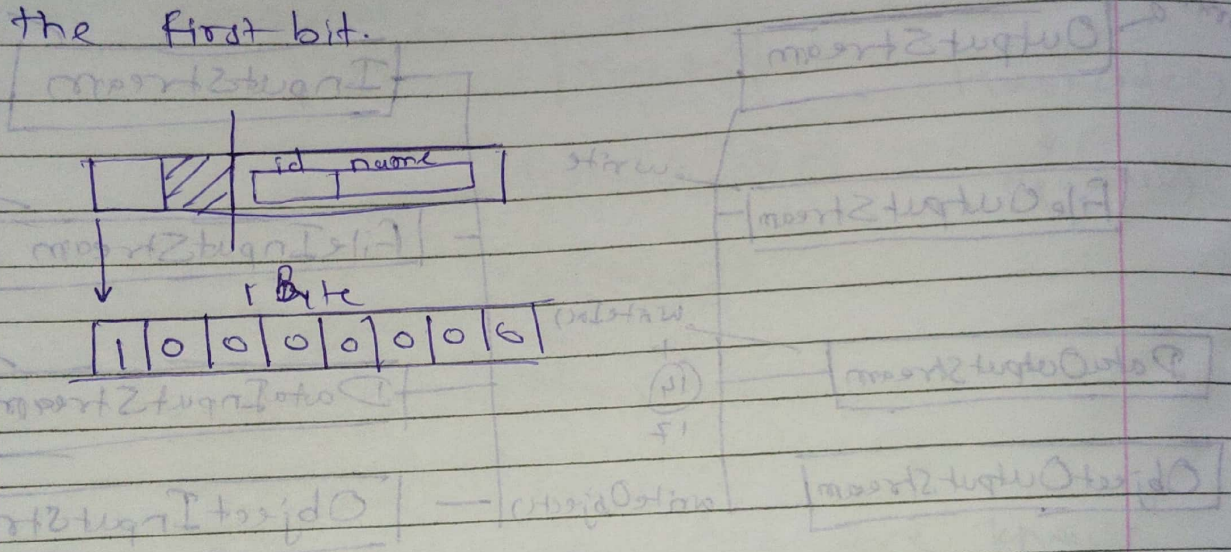


It mark first bit (by zero) to 1 which make ~~serial~~ serializable

Que What is markup interface

→ Markup interface is such interface which does not have any method to be implemented. It is also known as empty interface.

- Markup interface marks the bits internally to the object data following which JVM gets its meta data.
- Same way the serializable marks the first bit which is usually considered for the first bit.



a	action	result	decision	conclusion
	$a - b$	0	equals	no swap
	$a - b$	+ve	a is greater	swap
	$a - b$	-ve	b is greater	no swap